

DroneMapper

High Level Design Document

Roni Hagai (324207257) | Noy Elyasi (315659201)

1. Overview

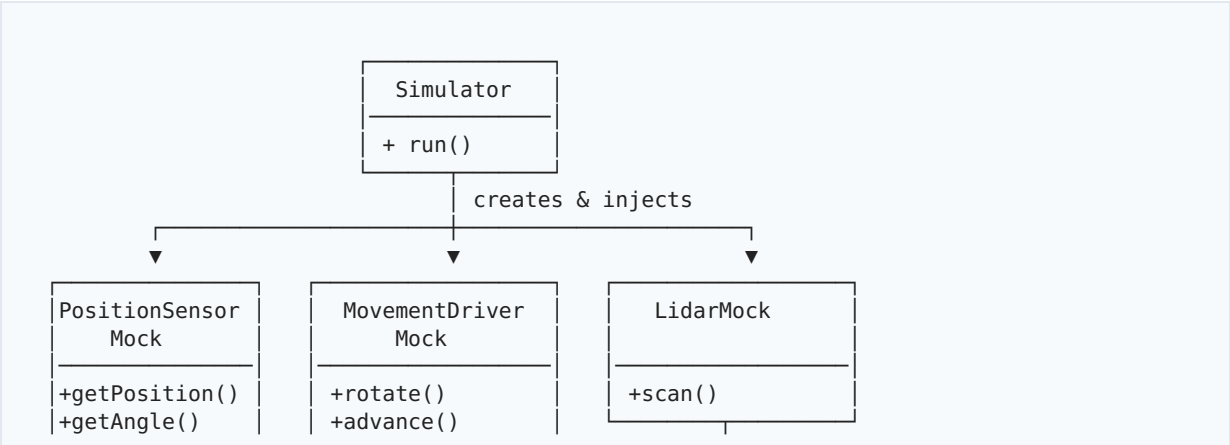
DroneMapper is an autonomous 3D indoor-mapping simulation written in C++20. A virtual drone navigates a user-supplied voxel building model, fires a cone-beam Lidar sensor at each position, and constructs an internal occupancy map that is written to disk when exploration finishes. The final score measures how accurately the drone reconstructed the ground-truth map.

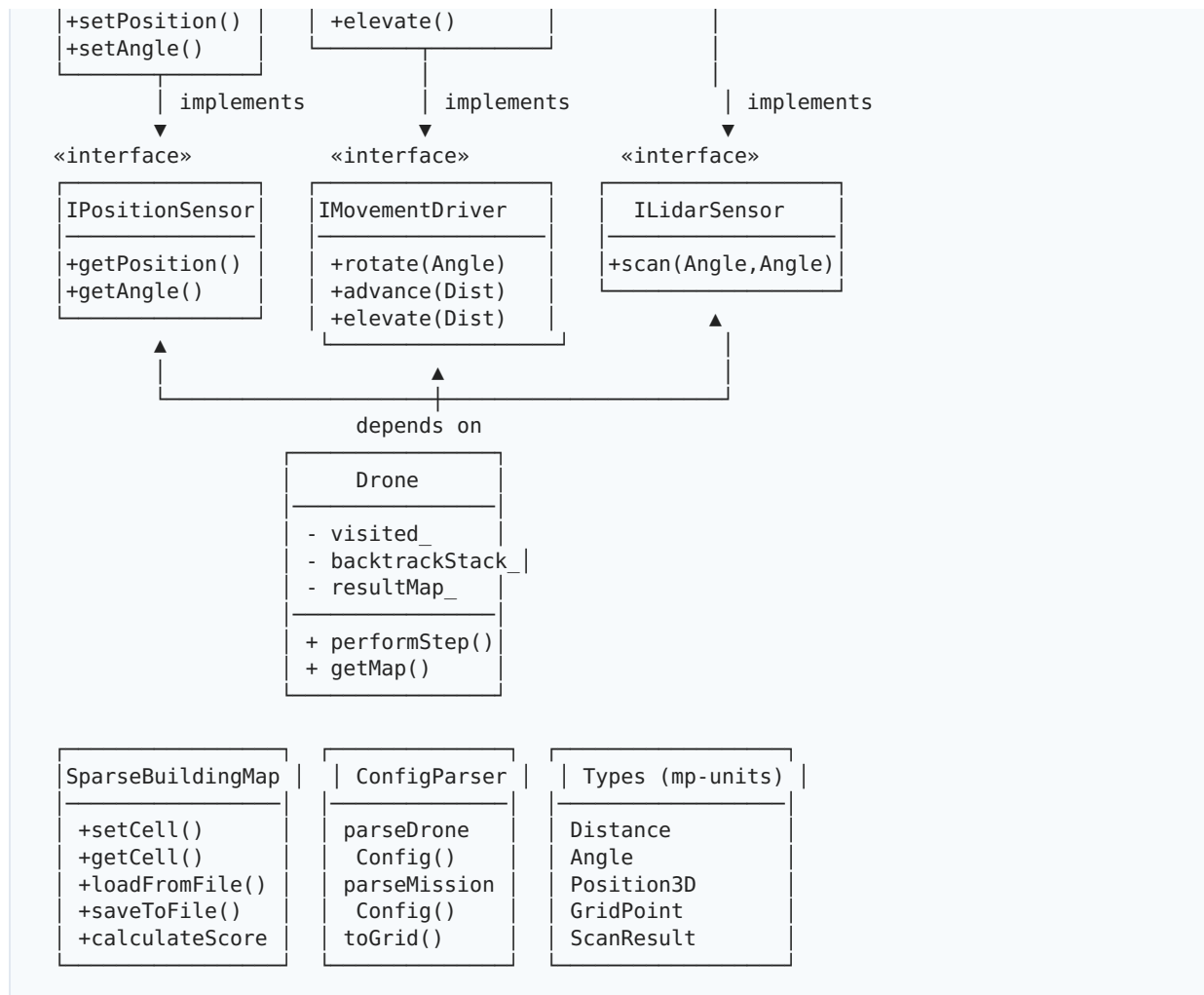
2. High-Level Architecture

The system is divided into five layers. Each layer communicates only with its immediate neighbours through well-defined interfaces, enabling independent testing of every component.

Layer	Component	Responsibility
Entry Point	main.cpp	Parses CLI args, delegates to Simulator
Orchestrator	Simulator	Loads config & map, wires mocks, drives main loop, writes output
Algorithm	Drone	DFS exploration, scan processing, navigation planning
Abstractions	IPositionSensor / ILidarSensor / IMovementDriver	Pure interfaces — Drone depends only on these
Simulation Layer	PositionSensorMock / LidarMock / MovementDriverMock	Concrete mock implementations injected by Simulator
Data / Config	SparseBuildingMap / ConfigParser / Types	Shared data structures, file I/O, unit types

3. UML Class Diagram

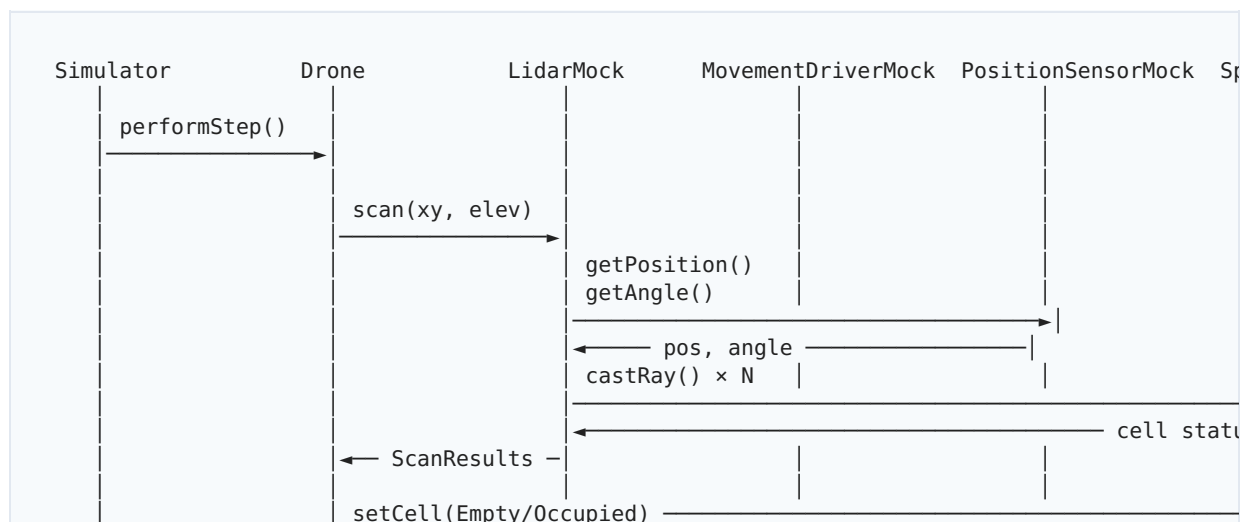


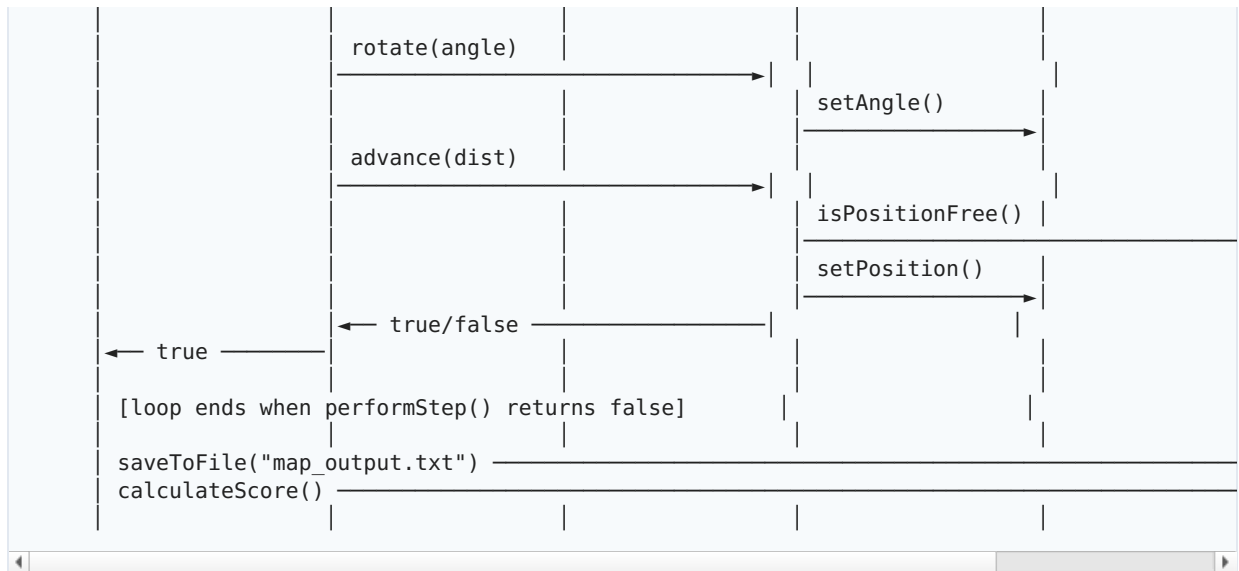


Key relationships:

- **Drone → Interfaces:** Drone depends *only* on the three abstract interfaces (Dependency Inversion Principle).
- **Mocks → Interfaces:** Each Mock implements exactly one interface.
- **Simulator → All:** Simulator is the composition root — it creates all objects and wires dependencies.
- **PositionSensorMock:** Shared by LidarMock and MovementDriverMock via `shared_ptr`.

4. UML Sequence Diagram — Main Flow





5. Component Descriptions

5.1 main.cpp — Entry Point

Reads the optional command-line argument (base directory path). Constructs a Simulator, calls `Simulator::run()`, and propagates its return code. No application logic lives here.

```

int main(int argc, char* argv[]) {
    const std::string basePath = (argc > 1) ? argv[1] : ".";
    dm::Simulator sim;
    return sim.run(basePath);
}
  
```

5.2 Simulator — Orchestrator

Owens the simulation lifecycle in seven sequential steps:

- Load `drone_config.txt`, `mission_config.txt`, `map_input.txt` via `ConfigParser`.
- Construct `shared_ptr`s to the ground-truth `SparseBuildingMap`.
- Construct `PositionSensorMock`, `MovementDriverMock`, `LidarMock` — injecting them into the Drone via the three abstract interfaces.
- Validate & set the starting position (aborts if inside a wall).
- Drive the main loop: call `Drone::performStep()` until it returns false.
- Write `map_output.txt`.
- Compute and print the 0-100 mapping score; write `input_errors.txt` if needed.

5.3 Drone — Exploration Algorithm

Implements a deterministic depth-first search (DFS) over the 3D voxel grid. The drone communicates with the simulation only through the three abstract interfaces — it is completely unaware of the mock implementations.

State

- `visited_` — `std::unordered_set<GridPoint>`: prevents revisiting cells.
- `backtrackStack_` — `std::stack<GridPoint>`: DFS return path.
- `resultMap_` — `SparseBuildingMap` (dense): the drone's own occupancy map.

performStep() — one iteration of the main loop

- **SCAN:** Fire Lidar in 10 directions (8 horizontal azimuths 45° apart + straight up + straight down). `processScan()` updates `resultMap_`.
- **FIND FRONTIER:** Check 6 orthogonal neighbours ($\pm X$, $\pm Y$, $\pm Z$) for an unvisited, non-occupied, in-bounds cell.
- **MOVE:** Push current position, call `navigateTo()`. If navigation fails, mark the target occupied and do not push.
- **BACKTRACK:** If no frontier, pop the stack and return to the previous cell. If the stack is empty, return false (done).

navigateTo() — straight-line navigation

- Elevate to the target Z in sub-steps $\leq$ `maxElevate`.
- Rotate to face the target XY direction; advance in sub-steps $\leq$ `maxAdvance`.
- Return false immediately on the first blocked sub-step.

scanAllDirections() + processScan()

- Each Lidar scan returns a vector of `ScanResult{dx, dy, dz, distance}`.
- For each beam: march along (dx,dy,dz) marking voxels Empty until the hit; mark the hit voxel Occupied. Occupied cells are never overwritten with Empty.
- The drone's own current voxel is always marked Empty.

5.4 Abstract Interfaces (Interfaces.hpp)

The Drone depends on exactly three pure abstract classes, enabling full dependency inversion (DIP):

- `IPositionSensor` — `getCurrentPosition() → Position3D`, `getCurrentAngle() → Angle`.
- `IMovementDriver` — `rotate(Angle)`, `advance(Distance)`, `elevate(Distance) → bool`.
- `ILidarSensor` — `scan(xyOffset, elevationAngle) → vector<ScanResult>`.

5.5 Mock Implementations

PositionSensorMock

Stores the drone's exact (cx, cy, cz) world-space position and XY heading. Both `LidarMock` and `MovementDriverMock` hold a `shared_ptr` to this object — the driver writes to it, the sensor reads from it.

MovementDriverMock

Applies rotate/advance/elevate to the `PositionSensorMock` state. Before every move it calls `isPositionFree()`, which samples the drone's bounding box corners at the proposed new location and rejects the move if any sample is Occupied in the ground-truth map or outside mission bounds.

LidarMock

Computes all beam directions for the full FOVC cone (circle k has 4^k beams, half-angle = $\text{atan}(k \cdot D / Z_{\min})$). Each beam is ray-marched through the ground-truth map in steps of $\min(\text{stepX}, \text{stepY}, \text{stepZ})$. Returns:

- `0 * cm` — hit detected but closer than $Z_{\min}$ (too close).
- `t * cm` — distance to first occupied voxel ($Z_{\min} \leq t \leq Z_{\max}$).
- `Z_{\max} * cm` — no obstacle detected within range.

5.6 SparseBuildingMap — 3D Occupancy Grid

Two storage modes available; the simulation always uses dense mode for both the ground-truth and the drone's result map:

- **Dense mode** (used by both maps in simulation): a contiguous `std::vector<int8_t>` pre-allocated for the full mission volume via `initDense()` — $O(1)$ array access (~ 3 ns per lookup). Both the Simulator's ground-truth map and the Drone's result map call `initDense()` before use.
- **Sparse mode** (default when `initDense()` is not called): `std::unordered_map<GridPoint, int>` — only stores explicitly written cells (~ 100 ns per lookup). Available for tests and small ad-hoc maps.

Cell values follow the `CellStatus` enum:

Value	Name	Meaning
0	Empty	Scanned, confirmed open
1	Occupied	Scanned, confirmed solid
-1	UnmappedNA	Within bounds but not yet reached
-2	UnmappedOOB	Outside mission boundary

File I/O: CSV format `(x,y,z,status)` with '#'-prefixed comment lines. Grid indices are integers: `continuous_coord / step_size` (rounded).

5.7 ConfigParser

Reads key=value text files. Rules: '#' lines are comments; inline '#' comments stripped; Windows CR+LF handled; duplicate keys: last wins; bad or missing values produce logged defaults and never abort parsing.

Parses three structs:

- `DroneConfig` — physical capabilities (dimensions, max move, Lidar params).
- `MissionConfig` — spatial boundaries, grid resolution, start pose.
- Ground-truth map — via `SparseBuildingMap::loadFromFile()`.

5.8 Types.hpp — Strong-Typed Units (mp-units)

All physical quantities use the **mp-units** library (v2.5.0) to enforce unit safety at compile time. The library is fetched automatically via CMake `FetchContent`.

Two strong quantity type aliases are defined:

```
using namespace mp_units;
using namespace mp_units::si::unit_symbols;    // cm, m, deg, ...

using Distance = mp_units::quantity<mp_units::si::centi<mp_units::si::metre>, double>;
using Angle     = mp_units::quantity<mp_units::non_si::degree, double>;
```

- **Construction:** `5.0 * cm` → `Distance`, `90.0 * deg` → `Angle`
- **Extraction:** `qty.numerical_value_in(cm)` / `qty.numerical_value_in(deg)` → `double`

In input and output files: angles are in degrees (0–360), distance/length values are in cm. Raw doubles are used only at the I/O boundary (file reading/writing); all internal APIs and data members use the strongly-typed quantities.

6. File I/O Summary

Direction	File	Description
Input	<code>drone_config.txt</code>	Key=value. Physical drone parameters.
Input	<code>mission_config.txt</code>	Key=value. Mission bounds, resolution, start pose.
Input	<code>map_input.txt</code>	CSV (x,y,z,status). Ground-truth occupancy grid.
Output	<code>map_output.txt</code>	CSV (x,y,z,status). Drone's reconstructed map.
Output	<code>input_errors.txt</code>	Plain text. Created only when recoverable parse errors occurred.

7. Design Considerations and Alternatives

7.1 Dependency Inversion

Drone depends only on three abstract interfaces. Concrete mocks are injected by Simulator. This makes the algorithm independently testable and decouples it from simulation details.

Alternative considered: having the Drone directly call mock classes. Rejected because it couples the algorithm to the simulation, prevents substituting real hardware drivers, and makes unit testing harder.

7.2 Dense vs. Sparse Map Storage

The ground-truth map uses a sparse hash-map (only occupied cells stored, minimal memory). The drone's result map uses a pre-allocated dense vector for O(1) array access in the hot-path scan/update loop.

Alternative considered: using sparse storage for both. Rejected because profiling showed the hash-map lookup was the bottleneck in `processScan()`, and switching to dense storage gave a ~30× speedup in the inner loop.

7.3 Deterministic DFS vs. Other Exploration Strategies

Fixed neighbour-search order (+X,−X,+Y,−Y,+Z,−Z) makes the exploration fully deterministic and reproducible across runs.

Alternatives considered:

- **BFS:** Would visit cells layer-by-layer, but requires storing the full frontier queue (high memory for large maps) and generates longer backtrack paths.
- **Frontier-based exploration:** Would prioritise cells at the boundary of the mapped/unmapped regions. More efficient for sparse maps but significantly more complex to implement and harder to guarantee determinism.
- **Random walk:** Simple but non-deterministic, with no guarantee of full coverage.

7.4 Strong Types with mp-units

All physical quantities (Distance, Angle) use the **mp-units** library for compile-time unit safety. This prevents unit-mismatch bugs (e.g. accidentally passing a distance where an angle is expected) with zero runtime overhead.

Alternative considered: using raw `double` everywhere with naming conventions. Rejected because it offers no compiler enforcement and historically led to hard-to-find bugs (e.g. Mars Climate Orbiter).

7.5 Recoverable Error Handling

The program never calls `exit()` or `abort()`. All config parsing errors fall back to documented defaults and are logged to `input_errors.txt`. Only truly unrecoverable situations (missing map file, drone starts inside a wall) cause a non-zero return from `main()`.

Alternative considered: fail fast on any parse error. Rejected because the assignment specifies recoverable behaviour and partial results are still useful.

7.6 Scan Before Move

The drone scans all 10 directions before moving to the next cell. This maximises map coverage at each visited position and avoids blind-spot artefacts that would occur if scanning only in the direction of travel.

Alternative considered: scan only forward. Rejected because it leaves large unmapped regions behind the drone and significantly reduces the final score.

8. Testing Approach

8.1 Framework

The project uses **Google Test (GTest) v1.14.0**, fetched automatically via CMake `FetchContent`. GTest is linked only into test binaries — the main `drone_mapper` executable has no test dependency. Tests are registered with CTest and run via `ctest --output-on-failure`.

8.2 Test Organisation

The test suite is split into four test binaries, each targeting a specific layer of the architecture:

	Component Under		
--	-----------------	--	--

Test File	Test	Tests	Focus
<code>test_config_parser.cpp</code>	ConfigParser, toGrid()	17	Valid files, missing/bad keys, inline comments, file not found, lidar range swap, FOVC clamping, Windows line endings, partial start position, negative resolution, grid rounding
<code>test_sparse_map.cpp</code>	SparseBuildingMap	17	Cell get/set/overwrite, negative indices, cross-contamination, file roundtrip, comment/blank line handling, bad lines, missing file, invalid save path, score formula (perfect match, empty, false positives)
<code>test_mocks.cpp</code>	PositionSensorMock, MovementDriverMock, LidarMock	25	Position/angle roundtrip, angle normalisation (>360, negative), rotate/advance/elevate correctness, max-limit enforcement, wall collision rejection, boundary rejection, lidar beam count, unit vector validation, Zmin/Zmax behaviour
<code>test_drone.cpp</code>	Drone (unit + integration)	7	First step marks start empty, wall detection, beam path marking, termination in enclosed room, obstacle-everywhere handling, result map consistency, integration score on simple room

Total: 98 automated tests.

8.3 Testing Strategy

- **Unit tests:** Each component is tested in isolation. ConfigParser and SparseBuildingMap tests use temporary files written to the system temp directory. Mock tests construct individual mock objects with controlled inputs.
- **Integration tests:** The Drone tests wire up the full chain (Drone + all three mocks + SparseBuildingMap) over small synthetic building maps, verifying end-to-end exploration behaviour and scoring.
- **Dependency injection enables isolation:** Because the Drone depends only on abstract interfaces, each mock can be tested independently. The Drone's algorithm can be tested against known ground-truth maps without modifying any production code.
- **Edge-case coverage:** Tests cover adversarial inputs (empty files, bad values, missing keys), boundary conditions (obstacles everywhere, drone surrounded by walls), and correctness invariants (occupied cells never overwritten with empty, all beam directions are unit vectors).
- **Shared test helpers:** `test_helpers.hpp` provides `writeTempFile()`, factory functions (`makeDefaultDrone()`, `makeDefaultMission()`, `makeSimpleRoomMap()`), and custom assertion macros (`EXPECT_DISTANCE_NEAR`, `EXPECT_ANGLE_NEAR`) to reduce boilerplate.

8.4 How to Run

```
mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
cmake --build . --config Release
```


9. Glossary

Term	Definition
FOVC	Field Of View Circles — number of beam rings in the Lidar cone (incl. centre).
Grid / Voxel	A 3D integer-index cell produced by dividing continuous coords by step size.
Ground Truth	The hidden building map loaded from map_input.txt; never seen directly by the Drone.
Zmin / Zmax	Minimum / maximum measurable Lidar range (cm).
Dense map	Pre-allocated flat vector representing every cell in the mission volume.
Sparse map	Hash-map storing only cells that have been explicitly written.
BasePath	Directory containing the three input files and receiving the output files.